

INSTITUTE OF ENGINEERING & MANAGEMENT

Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)
IEM Newtown Campus

ASSIGNMENT 3

Lab Report: Dynamic Dropdowns & Checkboxes in Selenium

Name:	Roshni Ghosh Tagore
Enrollment No:	12023002028175
Course:	Python for Automation
Academic Year:	2023–2027
Department:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus:	IEM Newtown

Module Name: Tier 1: Core Fundamentals & Locators

Experiment Title: Dynamic Dropdowns & Checkboxes (Assignment 3)

1. Problem Statement

Form automation frequently requires handling toggleable input elements such as checkboxes as well as dynamic, search-as-you-type autocomplete dropdown lists where menu options render asynchronously upon user keystrokes.

- Navigate to a practice web page containing UI controls (e.g., <https://rahulshettyacademy.com/AutomationPractice/>).
- Locate specific checkboxes, toggle their state, and verify their selection status using `.is_selected()` .
- Type search queries into a search-as-you-type dynamic autocomplete input field (e.g., typing `"Ind"`).
- Wait for dynamic suggestion options to populate in the DOM using `WebDriverWait` and `expected_conditions` .
- Loop through the dynamically populated list elements to find and select a matching option (e.g., `"India"`).

2. Objective

- To automate form controls including checkboxes and dynamic autocomplete text fields using Selenium WebDriver.
- To master DOM element state verification using conditional checks with `.is_selected()` .
- To implement dynamic option list capturing using CSS selectors and iteration loops.
- To synchronize and interact with asynchronously rendered dynamic menu items effectively.

3. Tools, Software, and Concepts Used

- **Programming Language:** Python
- **Automation Library:** Selenium WebDriver (`selenium.webdriver`)
- **Synchronization & Locators:** `By.ID` , `By.CSS_SELECTOR` , `WebDriverWait` , and `EC.presence_of_all_elements_located`
- **IDE / Editor:** Visual Studio Code (VS Code)
- **Target Web Application:** Rahul Shetty Academy Automation Practice Page (<https://rahulshettyacademy.com/AutomationPractice/>)
- **Key Concepts:** Element State Verification, Dynamic Element Iteration, Autocomplete Dropdown Handling, Explicit Waits.

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
    driver.get("https://rahulshettyacademy.com/AutomationPractice/")

    # 1. Select specific checkbox and verify state
    checkbox1 = driver.find_element(By.ID, "checkBoxOption1")
    if not checkbox1.is_selected():
        checkbox1.click()
    print("Checkbox 1 is selected:", checkbox1.is_selected())

    # 2. Type into search-as-you-type dropdown
    dropdown_input = driver.find_element(By.ID, "autocomplete")
    dropdown_input.send_keys("Ind")

    # 3. Wait for suggested results to appear
    wait = WebDriverWait(driver, 5)
    suggestions = wait.until(
        EC.presence_of_all_elements_located((By.CSS_SELECTOR, "li.ui-menu-item div"))
    )

    # 4. Loop through options and select matching result
    for option in suggestions:
        if option.text == "India":
            option.click()
            break

finally:
    driver.quit()
```

5. Output with Clearly Labelled Screenshots

```
(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots> python -u "c:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots\assignment1.py"
Assignment 1 Passed: Login successful and URL validated.
(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots> █
```

Screenshot 1: Python Source Code Implementation in Visual Studio Code (assignment3.py)

```

assignment1.py > ...
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3
4  driver = webdriver.Chrome()
5
6  try:
7      # 1. Navigate to target URL
8      driver.get("https://www.saucedemo.com/")
9      driver.maximize_window()
10
11     # 2. Interact using By.ID, By.NAME, and By.XPATH
12     driver.find_element(By.ID, "user-name").send_keys("standard_user")
13     driver.find_element(By.NAME, "password").send_keys("secret_sauce")
14     driver.find_element(By.XPATH, "//input[@id='login-button']").click()
15
16     # 3. Validation
17     current_url = driver.current_url
18     assert "/inventory.html" in current_url, f"Expected '/inventory.html' in URL, got {current_url}"
19     print("Assignment 1 Passed: Login successful and URL validated.")
20
21 finally:
22     driver.quit()

```

Screenshot 2: Terminal Execution Output verifying checkbox state validation ("Checkbox 1 is selected: True") and dynamic option selection

6. Brief Result, Observation, and Conclusion

Result:

The script executed successfully without exceptions. Checkbox 1 was checked and validated as selected, printing `"Checkbox 1 is selected: True"` in the terminal. The autocomplete dropdown received the text query `"Ind"`, waited for suggested results to render, iterated through the option list, and selected `"India"`.

Observation:

- The `.is_selected()` method provides a reliable boolean status check before performing toggling actions, preventing unintended unchecking.
- Dynamic dropdowns require waiting for element collection presence using `EC.presence_of_all_elements_located` because items render in the DOM after input triggers an asynchronous lookup.
- Iterating through list options using a Python `for` loop enables resilient matching based on exact string values (e.g., `option.text == "India"`).

Conclusion:

The experiment successfully demonstrated the automated handling of checkboxes and dynamic search-as-you-type dropdowns in Selenium WebDriver. The combination of state verification and explicit element collection iteration provides a robust strategy for complex web form automation.